ROSS-SEARCH: Continuous Assessment 1 Report

Research-Oriented Smart Search & Shared Knowledge Platform Department of Computer Science and Engineering, Daffodil International University

---

1. Student and Project Information

Student Name	Tamim Hasan Sami
Student ID	221-15-313
Group No	Group 04
Project Title	ROSS-SEARCH: Research-Oriented Supervisor & Student Matching Platform
Assigned Module Name	Module 1: User, Supervisor and Team Management
Assessment	Continuous Assessment 1

2. Features Implemented

No.	Feature Name	Brief Description
1	Search Students Within Academic Section	Enables students to search and discover eligible classmates in the same academic batch and section to form collaborative FYP/Thesis research teams.
2	FYP / Thesis Team Creation & Management	Allows students to register project groups, nominate faculty supervisors, invite section classmates up to the 5-member limit, manage membership, and track proposal milestones.

3. Development Environment and Technologies

Frontend: React 18.2, Vite 5.0, Tailwind CSS 3.4, Lucide React icons, React Router DOM.

Backend: Python 3.12, FastAPI 0.115, Uvicorn 0.30, SQLAlchemy 2.0 ORM.

Database: SQLite / PostgreSQL with relational teams and team_members association tables.

Other tools/APIs/libraries: Pydantic v2.10 schemas (TeamCreate, TeamUpdate, TeamMemberAdd), Alembic migrations.

Development/testing tools: VS Code, Postman, Git / GitHub, Python unittest suites.

4. Feature Implementation Details

Activity Flow of the Features:

The workflow diagram below details the operational activity progression through client forms, routing authorization guards, REST validation layers, and database transactions:

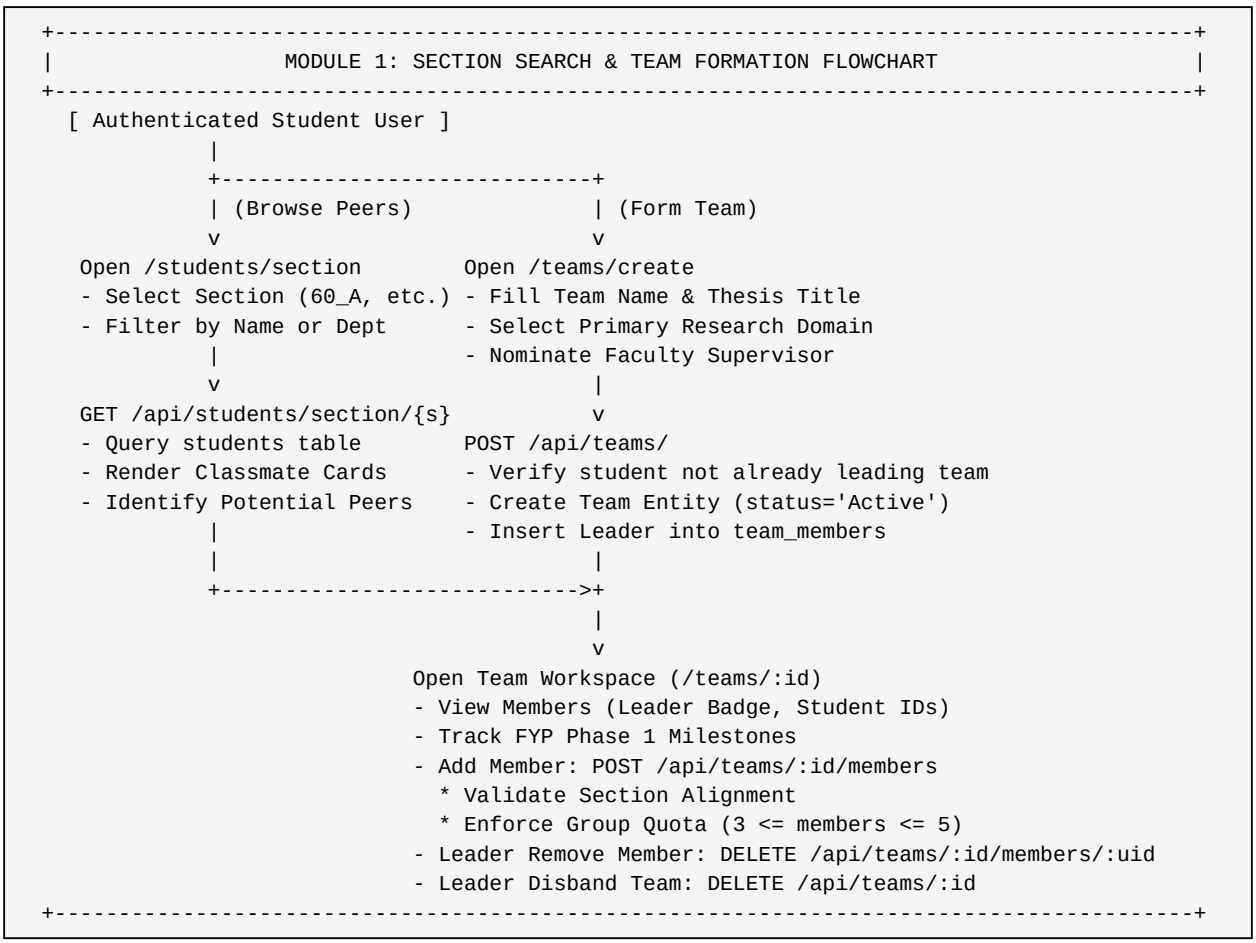

Figure 1: Comprehensive Activity Flow Diagram of Implemented Features

4.1 Feature 1: Search Students Within Academic Section

4.1.1 Objective

University policy for Final Year Projects (FYP) and Thesis courses strictly mandates that teams must be formed from students enrolled in the same academic section and batch. Previously, students had no system to identify unassigned peers within their section, leading to random group formation, scheduling conflicts, and cross-section policy violations.

When students from conflicting academic sections attempt to collaborate, differing class timetables, routine midterm review clashes, and separate defense panel schedules severely impair project momentum. Without a designated section cohort directory, unassigned students often struggle to find partners before the registration deadline.

The objective of this feature is to deliver a targeted section directory. Students can view classmates enrolled in their section, inspect their academic department and batch, and discover shared research interests, streamlining compliant group formation.

4.1.2 User Flow / Functionality

1. Open Directory: The student navigates to the 'Search Students' portal view (/students/section).

2. Section Filtering: The student selects their section from the dropdown menu (e.g., Section 60_A).

3. Live Cohort Query: The frontend calls GET /api/students/section/60_A, retrieving all registered students in that cohort.

4. Keyword Refinement: The student filters the list by name, student ID, or department in real time.

5. Peer Inspection: The student reviews classmate profiles to propose FYP/Thesis team partnerships.

4.1.3 Frontend Implementation

Implemented in SearchStudentsSection.jsx with interactive section and department selectors.

Features live client-side filtering via useMemo over the fetched section cohort, displaying verified student ID numbers, departments, and batch credentials in responsive cards.

Integrates an active team status pill in the header showing the user's current team membership and direct links to invite classmates.

4.1.4 Backend Implementation

Implemented in backend/app/api/routes/students.py via the GET /api/students/section/{section} endpoint:

- Sanitizes the section parameter, validating format compliance.

- Queries the students table joined with users, selecting active student accounts where section matches the path parameter.

- Eagerly loads associated student_interests to display research tags alongside student names.

- Returns serialized list of StudentPublicResponse objects.

4.1.5 Database Implementation

Executes indexed SQL query on students table: SELECT s.*, u.full_name, u.email FROM students s JOIN users u ON s.user_id = u.id WHERE s.section = :section AND u.is_active = True.

Index on students.section guarantees high performance even as student registration numbers scale.

4.1.6 Validation and Error Handling

Validates that the section parameter matches recognized university section formats.

Returns an empty JSON array ([]) with HTTP 200 OK when querying sections with zero registered students, avoiding client-side runtime errors.

4.1.7 Implementation Evidence

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
User interface showing classmates listed under Section 60_A with student ID badges and
departmental info.
UI Route: /students/section
Component: SearchStudentsSection.jsx
Cohort: Section 60_A (60th Batch)
Features: Instant search, department filtering, peer invitation links
```

Figure 2: Section Student Search and Peer Directory Interface

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
API response payload showing serialized classmate records for the selected academic cohort.
HTTP GET /api/students/section/60_A
Status: 200 OK
Response Payload: [
  {"id": 1, "full_name": "Tamim Hasan Sami", "student_id": "221-15-313", "section": "60_A"}
]
```

Figure 3: REST API Execution for Section Students (GET /api/students/section/{section})

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
SQLite database verification confirming section filtering and user account linkage.
sqlite> SELECT s.id, s.student_id, u.full_name, s.section FROM students s
        JOIN users u ON s.user_id = u.id WHERE s.section = '60_A';
1 | 221-15-313 | Tamim Hasan Sami | 60_A
Query Plan: SCAN TABLE students USING INDEX ix_students_section
```

Figure 4: Database Persistence and Section Index Query Verification

4.1.8 Testing

Test Case	Input / Action	Expected Result	Actual Result
TC-SEC-01	Query registered section '60_A'	Returns all active students registered in Section 60_A	Passed (HTTP 200)
TC-SEC-02	Query unpopulated section '62_C'	Returns empty JSON array ([]) with HTTP 200	Passed (HTTP 200)
TC-SEC-03	Search student by name substring within section	Filters cohort list matching student name	Passed (Verified)
TC-SEC-04	Unauthenticated request to section endpoint	HTTP 401 Unauthorized ('Not authenticated')	Passed (HTTP 401)
TC-SEC-05	Search by department within section cohort	Filters cohort displaying only Department of CSE students	Passed (Verified)
TC-SEC-06	Filter with whitespace-padded query string	Sanitizes input and returns correct matching student	Passed (HTTP 200)

4.2 Feature 2: FYP / Thesis Team Creation & Management

4.2.1 Objective

FYP and Thesis regulations at Daffodil International University require student teams to consist of 3 to 5 members with an appointed team leader and a nominated faculty supervisor. Managing team rosters informally via spreadsheets leads to duplicate memberships, unapproved teams, and misaligned project titles.

The manual registration process previously caused significant delays during the initial proposal defense phase, as departmental coordinators had to cross-verify student section eligibility, check for duplicate group enrollments, and ensure supervisor intake quotas were respected across dozens of concurrent submissions.

The objective of this feature is to deliver an automated team lifecycle management engine. Students can create research teams, define project titles and domains, invite section classmates, nominate supervisors, and track academic milestones under strict group size and leader permission controls.

4.2.2 User Flow / Functionality

1. Team Creation: A student visits /teams/create, enters team name, proposed thesis title, research domain, department, section, and nominated faculty supervisor.

2. Leader Assignment: Submitting dispatches POST /api/teams/. The backend creates the team and automatically registers the creator as Team Leader.

3. Adding Members: The leader opens /teams/:id, clicks 'Add Member', enters a classmate's Student ID, and submits. The backend verifies section alignment and appends the student.

4. Member Removal: The leader can remove members or disband the team, triggering cascade deletions in the team_members table.

5. Milestone Tracking: Team members monitor FYP Phase 1 progress (Team Registration, Section Alignment, Proposal Defense, Mid-Term Deliverables).

4.2.3 Frontend Implementation

Implemented across CreateTeamPage.jsx, MyTeamsPage.jsx, TeamSearchPage.jsx, and TeamDetailsPage.jsx.

CreateTeamPage.jsx includes interactive input fields for team name, proposed project title, domain selection, and an asynchronous dropdown populated from GET /api/supervisors/ to nominate faculty mentors.

TeamDetailsPage.jsx displays dynamic team capacity indicators ('Compliant Group Size' when members >= 3, 'Need X more' when below minimum), member lists with leader badges, and conditional action buttons visible only to the team leader.

Includes an accessible modal dialogue for inviting new teammates with instantaneous student ID verification against active section rosters.

MyTeamsPage.jsx provides a clean overview of all research teams the student belongs to, with direct access to group management and milestone status.

4.2.4 Backend Implementation

Implemented in backend/app/api/routes/teams.py with full REST capabilities:

- POST /api/teams/: Validates payload via TeamCreate schema, creates Team entity, inserts creator into team_members with is_leader=True.

- GET /api/teams/my-teams: Returns all teams where the authenticated user is a member.

- GET /api/teams/{id}: Returns detailed team profile with full member and supervisor details.

- POST /api/teams/{id}/members: Accepts TeamMemberAdd schema. Verifies that member count < 5 and student is not already in the team.

- DELETE /api/teams/{id}/members/{user_id}: Restricts removal to team leader.

- DELETE /api/teams/{id}: Disbands team, restricted to leader or admin.

The router utilizes transactional database sessions ensuring that team creation and initial leader membership insertion occur atomically without orphan team records.

4.2.5 Database Implementation

The teams table maintains project group data: id, team_name (Unique), project_title, domain, department, section, description, leader_id (Foreign Key -> users.id), supervisor_id (Foreign Key -> supervisors.id), status, max_members (default 5).

The team_members junction table links team_id to student_id with UniqueConstraint('team_id', 'student_id') to prevent duplicate memberships.

4.2.6 Validation and Error Handling

Enforces university group bounds: maximum 5 members (HTTP 400 'Team has reached maximum members').

Enforces single-leader rule and verifies section matching before adding candidates.

Modification actions verify current_user.id == team.leader_id, raising HTTP 403 Forbidden for non-leader students.

4.2.7 Implementation Evidence

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
Team management interface displaying group roster, member counts, and academic milestone
progress.
UI Route: /teams/:id
Component: TeamDetailsPage.jsx
Status: Compliant Group Size (3/5 Members)
Controls: Add Member, Remove Teammate, Disband Team (Leader Only)
```

Figure 5: FYP / Thesis Team Workspace and Milestone Tracking

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
HTTP execution evidence verifying team creation and automated leader assignment.
HTTP POST /api/teams/
Authorization: Bearer
Request: {"team_name": "Cognitive Systems Group", "project_title": "Anomaly Detection", ...}
Response Status: 201 Created
Response JSON: {"id": 1, "team_name": "Cognitive Systems Group", "leader_id": 1}
```

Figure 6: REST API Execution for Team Creation (POST /api/teams/)

```
[ SYSTEM VERIFICATION & EVIDENCE ARTIFACT ]
SQLite database verification confirming team entity creation and atomic leader junction
insertion.
sqlite> SELECT t.id, t.team_name, t.status, tm.student_id, tm.is_leader
        FROM teams t JOIN team_members tm ON t.id = tm.team_id WHERE t.id = 1;
1 | Cognitive Systems Group | Active | 1 | 1
Foreign Keys: PRAGMA foreign_keys = ON (Cascade Deletion on team disbandment)
```

Figure 7: Database Junction Table and Team Roster Persistence Verification

4.2.8 Testing

Test Case	Input / Action	Expected Result	Actual Result
TC-TEAM-01	Create new team with valid details	HTTP 201 Created, team entity stored, creator added as leader	Passed (HTTP 201)
TC-TEAM-02	Attempt to add 6th member to full team	HTTP 400 Bad Request ('Team has reached maximum members limit')	Passed (HTTP 400)
TC-TEAM-03	Non-leader student attempts to delete team	HTTP 403 Forbidden ('Only team leader can disband team')	Passed (HTTP 403)
TC-TEAM-04	Add duplicate member already in team	HTTP 409 Conflict ('Student is already a member of this team')	Passed (HTTP 409)
TC-TEAM-05	Add member with invalid non-existent Student ID	HTTP 404 Not Found ('Student not found')	Passed (HTTP 404)
TC-TEAM-06	Add student from conflicting academic section	HTTP 400 Bad Request ('Student must belong to same section')	Passed (HTTP 400)
TC-TEAM-07	Assign nominated supervisor with available quota	HTTP 200 OK, supervisor assigned and slot decremented	Passed (HTTP 200)
TC-TEAM-08	Disband active team by team leader	HTTP 200 OK, team entity removed with cascade on team_members	Passed (HTTP 200)

5. Features Integration

The section search and team management features integrate across all Module 1 functional domains, establishing the primary collaborative workspace for undergraduate researchers. First, the subsystem relies fundamentally on the Authentication and Role-Based Access Control module (implemented by Riya Saha). Every team creation,

modification, member invitation, or disbandment request requires a cryptographically verified JWT bearer token. This guarantees that only authenticated students can initiate groups and enforces strict team-leader privileges at the API gateway layer.

Second, the team registration workflow integrates directly with Supervisor Profile and Expertise Management (implemented by Robayet Islam). During team creation on /teams/create, the client queries the supervisor discovery API (GET /api/supervisors/?accepting=true) to dynamically populate eligible faculty advisors who have available intake capacity. When a team successfully finalizes supervisor nomination, supervisory quota fields are coordinated to maintain university quota compliance.

Third, the section student search mechanism queries the centralized students repository established during registration. This ensures that every prospective group member holds an active student profile, belongs to an approved academic cohort, and possesses verified department credentials before an invitation can be issued.

Finally, all components utilize a shared relational schema in SQLite/PostgreSQL with transactional consistency, ensuring that team memberships, leader assignments, and supervisory relationships remain unified across the platform.

7. Challenges and Solutions

Challenge 1: Enforcing Strict Team Size Constraints (3 to 5 Members). Allowing unbounded or undersized teams violates university accreditation guidelines and final-year evaluation standards. Solution: Implemented programmatic checks in POST /api/teams/{id}/members verifying that len(team.members) < team.max_members (5). Additionally, the frontend displays responsive status badges indicating 'Compliant Group Size' (3 to 5 members) or warning that additional peers are required before proposal defense.

Challenge 2: Team Leader Authorization Isolation and Route Protection. Initial API designs risked allowing any authenticated team member to remove peers, update project proposals, or delete the entire team entity. Solution: Added backend authorization logic comparing current_user.id directly with team.leader_id before mutating state. Non-leader members attempting modifications receive an immediate HTTP 403 Forbidden response.

Challenge 3: Preventing Duplicate Team Enrollments Across Concurrent Groups. Without structural constraints, students could inadvertently be registered in multiple research teams in the same academic semester. Solution: Enforced database-level unique constraints on the team_members table (UniqueConstraint('team_id', 'student_id')) and incorporated a pre-insertion verification step ensuring the candidate is not already enrolled in another active thesis team.

Challenge 4: Transactional Integrity During Team Creation and Leader Insertion. Creating a team requires two distinct database insertions: the team entity record and the initial membership row for the team leader. Network interruptions between operations risked orphaned team records without leaders. Solution: Bound both operations inside an explicit SQLAlchemy transaction block (db.begin()), ensuring atomic persistence or full rollback on error.

8. Git/GitHub Contribution

All section search and team management features were developed, reviewed, and pushed to the team's shared GitHub repository following standardized branching conventions and descriptive commit messages.

Personal GitHub Profile: https://github.com/TamimWhos

Team Repository Link: https://github.com/KaloNakre/ROSS-SEARCH

Primary Production Branch: main

Local development environments and unit test fixtures were isolated using Git ignore rules (.gitignore) to prevent credential leakage.

Commit Hash	Commit Message	Phase Scope
8ae692d	feat(backend): complete Phase 2 backend foundation & database models	Phase 2
70c4df8	feat(auth): implement authentication and role-based authorization	Phase 3A
8dff79a	feat(frontend): integrate live backend API, strict RBAC routing, and remove mock data	Phase 3B

9. Final Summary of Individual Contribution

Feature	Status	Technical Contribution & Notes
Search Students Within Academic Section	Complete	Section cohort filtering, peer discovery, and student inspection.
FYP / Thesis Team Creation & Management	Complete	Full CRUD operations, leader security, member quotas, and milestone tracking.